

Start coding or [generate](#) with AI.

Mastering TensorFlow: Day 11 - Advanced Data Preprocessing with TensorFlow

```
import tensorflow as tf
import numpy as np

# Creating a sample dataset
data = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0]])

# Converting to a TensorFlow dataset
dataset = tf.data.Dataset.from_tensor_slices(data)

# Applying feature scaling (min-max normalization)
def scale_features(x):
    min_val = tf.reduce_min(x)
    max_val = tf.reduce_max(x)
    scaled_x = (x - min_val) / (max_val - min_val)
    return scaled_x

# Mapping the scaling function to the dataset
scaled_dataset = dataset.map(scale_features)

# Inspecting the scaled data
for element in scaled_dataset:
    print(element)
```



```
tf.Tensor([0.  0.5 1. ], shape=(3,), dtype=float64)
tf.Tensor([0.  0.5 1. ], shape=(3,), dtype=float64)
tf.Tensor([0.  0.5 1. ], shape=(3,), dtype=float64)
```

Feature Scaling: Normalizing data ensures that features contribute equally to model training.

```
# Sample categorical data
categories = tf.constant(['apple', 'banana', 'cherry'])

# One-hot encoding the categories
one_hot_encoded = tf.one_hot(tf.range(len(categories)), depth=len(categories))

# Inspecting the one-hot encoded data
print("One-Hot Encoded Data:\n", one_hot_encoded)
```

```
One-Hot Encoded Data:  
tf.Tensor(  
[[1. 0. 0.]  
 [0. 1. 0.]  
 [0. 0. 1.]], shape=(3, 3), dtype=float32)
```

One-Hot Encoding: Converts categorical data into a binary matrix, making it suitable for training machine learning models.

```
import tensorflow as tf  
import tensorflow.keras.layers as layers  
import numpy as np  
  
# Sample image (randomly generated for demonstration)  
image = tf.random.uniform(shape=[28, 28, 1], minval=0, maxval=255, dtype=tf.float32)  
  
# Define an augmentation layer  
data_augmentation = tf.keras.Sequential([  
    layers.RandomFlip("horizontal_and_vertical"),  
    layers.RandomRotation(0.2),  
)  
  
# Apply the augmentation  
augmented_image = data_augmentation(tf.expand_dims(image, 0))  
  
# Inspecting the augmented image  
print("Original Image Shape:", image.shape)  
print("Augmented Image Shape:", augmented_image.shape)
```

```
Original Image Shape: (28, 28, 1)  
Augmented Image Shape: (1, 28, 28, 1)
```

Image Augmentation: Applying transformations like flipping and rotation helps improve model generalization by creating variations in the training data.

Practical Tips for Data Preprocessing

1. **Feature Scaling:** Always scale your features before feeding them into your model, especially when working with distance-based algorithms.
2. **One-Hot Encoding:** Essential for categorical data when using algorithms that cannot work with string labels.
3. **Image Augmentation:** Improves model robustness by artificially increasing the size of your training dataset.

✓ Conclusion

Advanced data preprocessing is key to achieving better performance in your machine learning models. By scaling features, encoding categories, and augmenting images, you set the stage for effective model training. Today's session focused on these crucial steps, setting us up for success in the upcoming tasks.

Start coding or [generate](#) with AI.